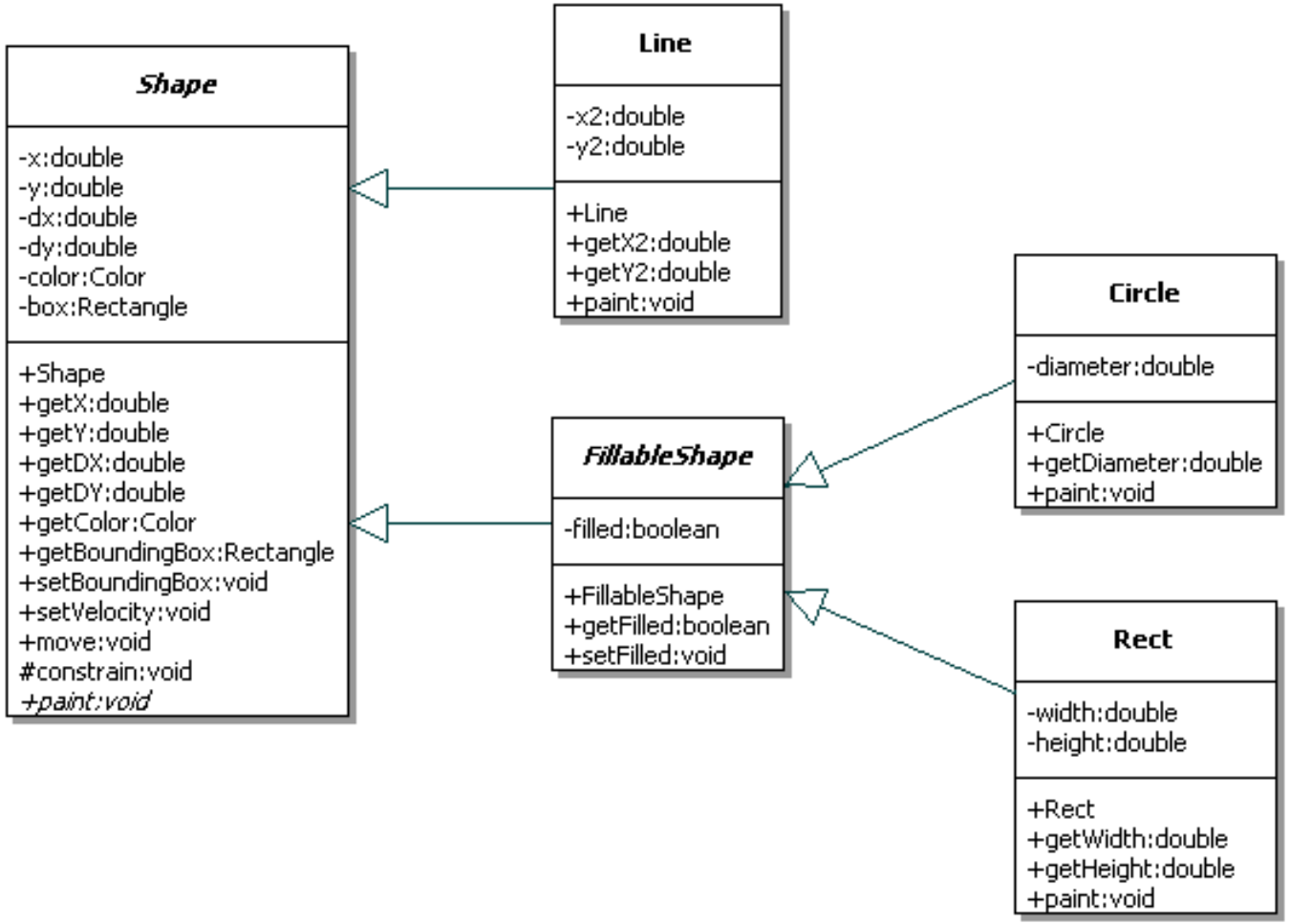
Arv och polymorfism

Inheritance (arv) – en påminnelse



- Hur många datamedlemmar, och vilka, har klassen Circle?
- Metoder i Circle?

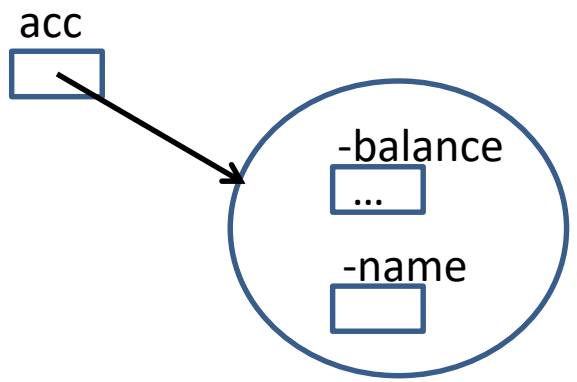
Inheritance (arv) - en påminnelse

Du måste veta vad dessa begrepp innebär, alternativt förstå, följande innan du går vidare

- extends
- protected
- Vad ärvs? Ärvs inte?
- super
- Overriding
  - som ju *inte* är detsamma som overloading
- När är det lämpligt med arv, när är det inte lämpligt?
  - Ex. vi har ArrayList och ska bygga Queue – arv eller komposition?

Referenser vs objekt, ännu en påminnelse

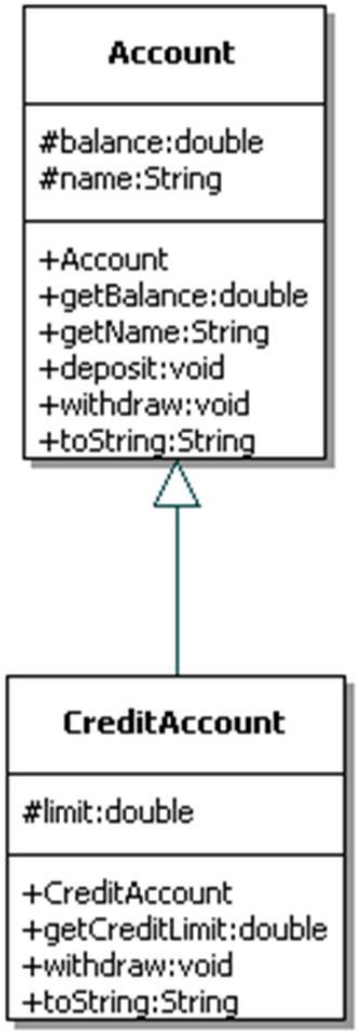
- Account acc;
- acc = new Account(”Anders”);



- ”acc” är en referensvariabel
  - Kan referera (”adressera”) ett objekt, eller…
  - …ha värdet ”null”
- ”new” tillsammans med konstruktor skapar ett nytt objekt och initierar detta
- *Vi kan tala om datatyper både för referenser och för objekt*
  - Typen ”referens till objekt av typen Account”
  - Typen Account (för själva objektet)

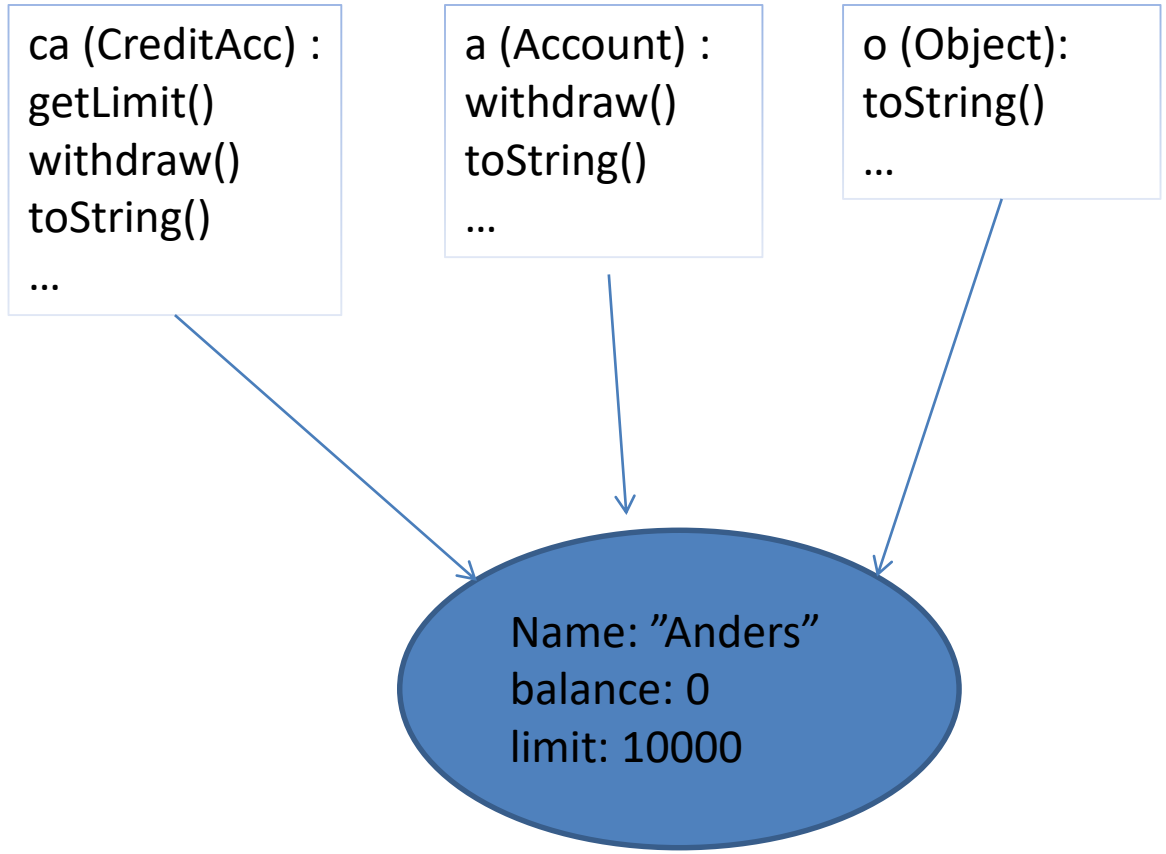
Referenser vs objekt vid arv

- En referens av typ T får referera objekt av typ T men även *referera objekt av subtyper till T*
  - Med andra ord: Objekt av typen T får refereras via referenser av typen T eller av referenser av supertyp
  - Här: En referens av typen Account får referera ett objekt av subtypen CreditAccount
- För en referens av given typ kan *endast metoder som existerar för referensens typ anropas*
  - Detta oavsett vilken typ det refererade objektet har
  - Kontrollen av att metoden existerar för referensens typ görs av kompilatorn, ”compile time”



Referenser vs objekt vid arv

- `CreditAccount ca = new CreditAccount(”Anders”, 10000);`
- Från referensen ”ca” ser vi objektet via gränssnittet för typen CreditAccount
- `Account a = ca; // ”cast” of` *reference*
- Från referensen ”a” ser vi objektet via gränssnittet för typen Account
- `Object o = ca;`
- Från referensen ”o” ser vi objektet via gränssnittet för typen Object



NB! Objektet är fortfarande detsamma, av sub-typen CreditAccount

Downcast av referenstyp

- `Account a = new CreditAccount(…);`
  - Om vi nu vill anropa metoder som är specifika för subtypen, t.ex. getCreditLimit?
- `CreditAcc ca = (CreditAcc) a;`
  - Vi kan göra ett ”downcast” av *referensen*
  - Förutsätter att det refererade objektet är av rätt typ (här CreditAcc)
  - Kontroll av det refererade objektets typ görs vid *exekveringen*, ”run time”
- Undvik exekveringsfel genom att först kontrollera objektets typ med operatorn instanceof
  - ```
    if(o instanceof CreditAcc) {
      CreditAcc ca = (CreditAcc) a; // downcast of reference
      double limit = ca.getCreditLimit();
    }
    ```
    ```

Polymorfism

- Polymorfism betyder ungefär ”många former”. Här: Många former för objekt och metoder
  - Ex: ”A + B” utförs på olika sätt om operanderna är av typen int, double respektive String
- Gemensamt (del-)gränssnitt för typer i en arvshierarki, men en viss operation kan utföras på olika sätt beroende på objektets typ …
- … eftersom en metod, i en arvshierarki, kan ha olika versioner i olika (sub-) klasser via *omdefiniering* – ”overriding”

Polymorfism och dynamisk bindning

- Metoden withdraw är omdefinierad i sub-klassen CreditAccount, dvs. finns i flera versioner, ”flera former”

- ```
  Account a1 = new Account(…);
  a1.withdraw(1000);
  ```
  – withdraw i klassen Account exekveras

- ```
  Account a2 = new CreditAccount(…);
  a2.withdraw(1000);
  ```
  – withdraw i klassen CreditAccount exekveras – *objektets typ* bestämmer vilken metod-version som exekveras

- I Java, och många andra oo-språk, exekveras alltid den metodversion som svarar mot *objektets typ*, via sk. dynamisk bindning
  – Kontrollen sker vid *exekvering*, ”run time”
  – Javas virtuella maskin kontrollerar det refererade objektets typ innan anropet exekveras
  – Påverkar prestanda, men ofta marginellt

Polymorfism – finns det någon poäng?

- Ja! Vi kan nu skriva kod där objektens typ inte är kända förrän vid exekveringen, ”run time”
- Förutsätter att typerna ingår i en arvshierarki
- När denna kod skrivs vet vi inte vilka objekt som kommer att skapas(!)
  - Vi vet bara att de är av typen Account eller subtyper till Account

```
// References of super type
Account[] list = new Account[100];
int n = 0;
...

// user decides type of object
if(userResponse.equals(”credit”) {
    list[n++] = new CreditAccount(userName, 1000);
} else {
    list[n++] = new Account(userName);
}
…

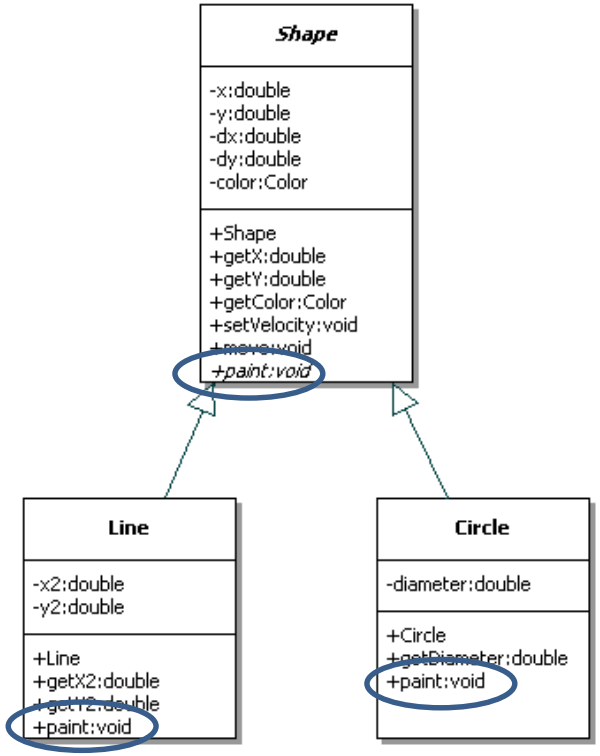
for(int i = 0; i < n; i++) {
   list[i].withdraw(100);
   System.out.println(list[i].toString());
}
```

Polymorfism och ”abstrakta typer”

- Polymorfism fungerar även för abstrakta klasser och interface
- Det är inte tillåtet att skapa objekt av abstrakt- eller interface-typ …
- … men tillåtet att skapa referenser av abstrakt- eller interface-typ …
- … som kan referera objekt av konkreta subtyper
  - Vi kan alltså skapa heterogena samlingar av objekt och samtidigt vara säkra på att rätt version av omdefinierade metoder exekveras

Polymorfism och ”abstrakta typer”

- Ex: Abstrakta klassen Shape …
- … med abstrakta metoden paint




```
// References of *abstract* type, Shape
ArrayList<Shape> shapes = new ArrayList<>();

// Objects of "concrete" subtypes Circle, Line, ...
shapes.add(new Circle(0.0, 0.0, 10.0));
shapes.add(new Line(0.0, 0.0, 20.0, 20.0));

// move all objects and repaint
for(int i = 0; i < shapes.size(); i++ ) {
   shapes.get(i).move();
   shapes.get(i).paint();  // polymorph code!
}
```